Bangladesh University of Engineering and Technology

Department of Computer Science and Engineering

AVL Trees and Interval Scheduling

CSE 208 — Data Structures and Algorithms II Sessional

Submission Deadline

10:30 PM, August 21, 2026

1 AVL Trees and Interval Scheduling

A **binary search tree** (BST) stores keys so that every key in a node's left subtree is smaller than the node's key and every key in its right subtree is larger. Search, insertion, and deletion follow a single root-to-leaf path and therefore take $O(h)$ time, where h is the tree height. An unfortunate insertion order can turn an ordinary BST into a chain with $h = n - 1$, making these operations $O(n)$.
A **balanced BST** adds a structural invariant. This keeps the height at $h = \Theta(\log n)$. Updates begin as ordinary BST operations and are followed by a small number of local repairs. A **rotation** changes parent-child relationships while preserving the in-order sequence of keys. Consequently, search remains identical to BST search, while insertion and deletion retain logarithmic worst-case time.
In this assignment, you will first implement an **AVL tree** and then augment the same balancing ideas to build a dynamic interval scheduling and conflict-query system.

1.1 AVL Trees

An **AVL tree** is a BST in which each node stores its height. For a node v, define

$$\mathrm{BF}(v) = \mathrm{height}(v.\mathrm{left}) - \mathrm{height}(v.\mathrm{right})$$

where $\mathrm{height}(u)$ is the height of the subtree rooted at u. The AVL invariant requires $\mathrm{BF}(v) \in \{-1, 0, 1\}$ for every node. If an update breaks this invariant, one of four imbalances occurs: left-left (LL), right-right (RR), left-right (LR), or right-left (RL). LL and RR use one rotation; LR and RL use two.
Insertion can stop after the first effective repair, although heights must remain correct.

Deletion begins with the ordinary BST procedure, but the physically removed node may reduce the height of its former subtree. The repair therefore proceeds as follows:

1. Delete the key as in a BST. For a node with two children, replace its key with its in-order successor, then physically remove that replacement node.
2. Starting at the parent of the physically removed node, move toward the root. At each node, recompute its height and balance factor.

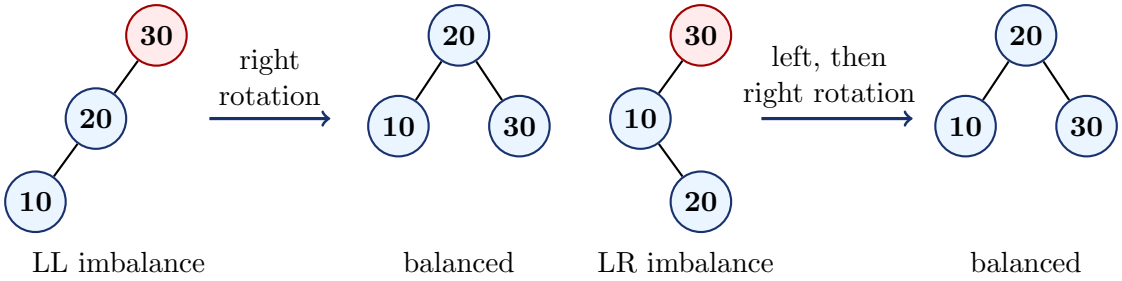


Figure 1: AVL repair examples. RR and RL are the mirror images of LL and LR.

3. If the balance factor becomes $+2$ or -2, select the single or double rotation from the heavy child's balance factor. Unlike insertion, the heavy child's balance factor may be 0; this still requires a single rotation.
4. Continue to the root, because one deletion can unbalance more than one ancestor. The total work remains $O(\log n)$.

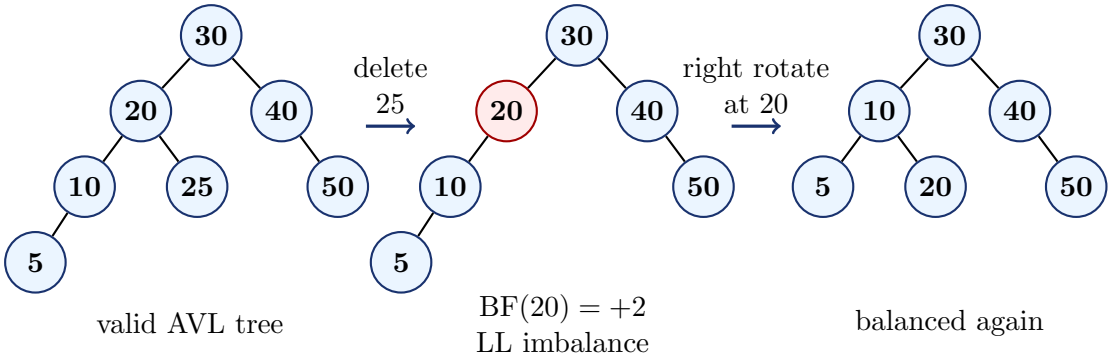


Figure 2: AVL deletion example: deleting 25 shortens the right subtree of node 20, producing an LL imbalance that is repaired by a right rotation.

1.2 AVL-Based Interval Scheduling

A scheduled event has an automatically generated integer identifier and a half-open interval $[s, e)$, where $s < e$. Identifiers begin at 1 and increase by 1 for every `ADD`; a removed identifier is never reused. Half-open intervals make endpoints unambiguous: $[10, 15)$ and $[15, 20)$ do not overlap. Two intervals A and B overlap exactly when

$$A.s < B.e \qquad \text{and} \qquad B.s < A.e.$$

Stored intervals are allowed to overlap; the application must find conflicts efficiently.

Store events in an AVL tree ordered lexicographically by $(\text{start}, \text{id})$. Thus, events with equal start times remain distinct and have a deterministic order. In addition to its height, every interval node stores

$$\text{maxEnd}(v) = \max(v.e, \text{maxEnd}(v.\text{left}), \text{maxEnd}(v.\text{right}))\,,$$

with $\text{maxEnd}(\text{empty}) = -\infty$. This value is the largest ending time anywhere in the node's subtree. It must be recomputed after every insertion, deletion, and rotation, just like height.

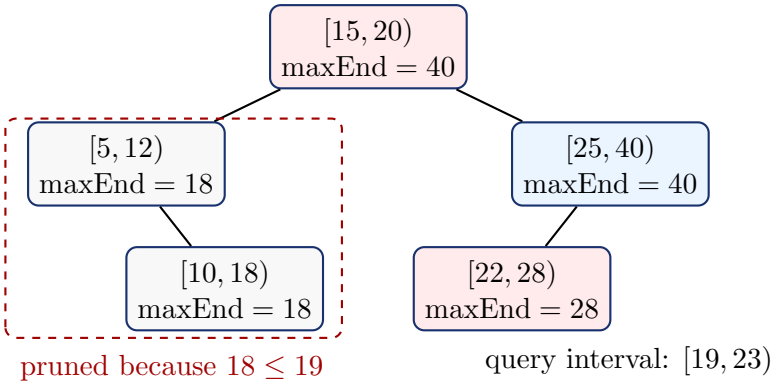


Figure 3: An interval query uses maxEnd to prune subtrees. The query $[19, 23)$ overlaps $[15, 20)$ and $[22, 28)$, while the entire left subtree can be skipped.

A conflict search follows one promising root-to-leaf path. To report all conflicts, perform a pruned in-order traversal: do not enter a subtree whose maxEnd is at most the query start, and do not inspect nodes whose start time is at least the query end unless an earlier-starting node may still exist in the left subtree. In-order reporting produces events in increasing $(\text{start}, \text{id})$ order. A point query at time t uses $s \leq t < e$, and finding the next event is an ordinary lower-bound search on the start time.

Interval operation	**Expected time**
Add, Remove, or Update	$O(\log n)$
Find any conflict or the next event	$O(\log n)$
Report all overlaps or events at a time	Pruned traversal; $O(n)$ worst case

Implementation Restriction

The tree algorithms must be your own implementation. You may use basic utilities such as `vector`, strings, streams, and timers, but you may not use `std::set`, `std::map`, policy-based ordered trees, or any library that already implements a balanced search tree.

2 Requirements

2.1 Language and source files

Use **C++** or **Java**. Submit two separately runnable programs:

- `AVLTree.cpp`, or `AVLTree.java`: the ordinary integer-key AVL implementation, its input/output code, and its timing code; and
- `IntervalScheduler.cpp`, or `IntervalScheduler.java`: the augmented interval AVL tree, its scheduling queries, its input/output code, and its timing code.

Custom header or helper files are allowed. You may reuse your AVL rotations and height-maintenance logic in the interval scheduler through shared helpers, templates, inheritance, composition, or another clean design. A generalized driver or run-time mode selector is not required.

The ordinary AVL implementation must provide this logical interface:

Method	Required behavior
`bool insert(int key)`	Insert a new key and return `true`; return `false` for a duplicate.
`bool erase(int key)`	Delete an existing key and return `true`; return `false` if absent.
`bool find(int key)`	Return whether the key is currently stored.
`vector<int> traverse()`	Return every stored key once, in strictly increasing order.

AVL keys and interval endpoints are signed 32-bit integers. Event IDs are generated internally as the positive sequence $1, 2, 3, \ldots$. All supplied intervals satisfy $s < e$, and all input files contain only syntactically valid commands. Duplicate AVL insertion and removal of missing data must leave the relevant tree unchanged. You are responsible for releasing all dynamically allocated memory.

2.2 Program execution

Compile and run the two programs separately. Each program accepts an input-file path and an operation-output-file path:

```
./avl_tree <input-file> <output-file>
./interval_scheduler <input-file> <output-file>
```

For example:

```
./avl_tree testcase.txt output_avl.txt
./interval_scheduler interval_testcase_basic.txt output_interval.txt
```

2.3 AVL Tree Implementation: Input and Output

The AVL input is a plain-text file with one operation per line:

Command	Meaning
`I x`	Insert integer x.
`D x`	Delete integer x, if present.
`F x`	Find integer x.
`T`	Traverse the current tree in order.

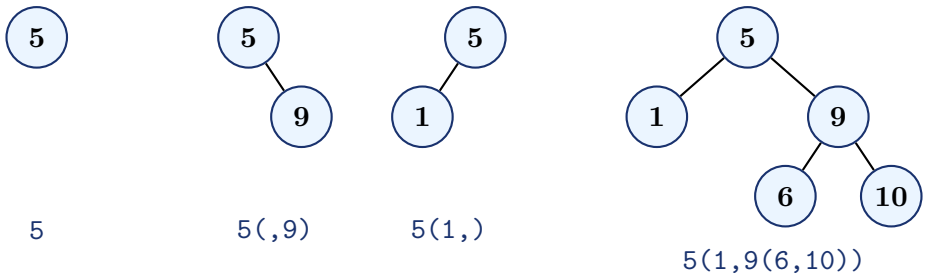
Write exactly one line per AVL command:

- `I x`: after a successful insertion, print the resulting tree in nested-parentheses format; print `duplicate` if x already exists;
- `D x`: after a successful deletion, print the resulting tree in nested-parentheses format; print `not found` if x does not exist;

- `F x`: print `found` or `not found`; and
- `T`: print the sorted keys separated by one space, with no trailing space. Print an empty line for an empty tree.

Define the nested-parentheses representation recursively:

- an empty subtree is represented by an empty string;
- a leaf is represented by its key only, for example `5`; and
- a non-leaf node is represented as `key(left,right)`. Both child positions and the comma must be present, even when one child is empty.



Do not print node heights. If a successful deletion makes the tree empty, print an empty line.

Sample AVL input

```
F 8
I 8
I 3
I 10
I 3
T
D 8
F 8
```

Sample AVL output

```
not found
8
8(3,)
8(3,10)
duplicate
3 8 10
10(3,)
not found
```

2.4 Interval Scheduling: Input and Output

The tree is ordered by (start, id). The scheduler maintains a `nextId` counter, initially 1. Every `ADD` operation assigns the current value of `nextId` to the new event and then increments the counter. Generated IDs are never reused, even after an event is removed. Several events may have identical intervals or start times. You may use `std::unordered_map` in C++ or `HashMap` in Java only to map an event ID to its current (start, end) pair. This hash table does not replace the interval AVL tree.
The input is a plain-text file with one operation per line:

Command	Meaning
`ADD s e`	Add an event with interval $[s, e)$ and assign the next generated ID.
`REMOVE id`	Remove the event with this ID.
`UPDATE id s e`	Replace the event's old interval with $[s, e)$.
`CONFLICT s e`	Determine whether any stored event overlaps $[s, e)$.
`OVERLAPS s e`	Report every event that overlaps $[s, e)$.
`AT t`	Report every event satisfying $s \leq t < e$.
`NEXT t`	Report the event with the smallest (start, id) among those whose start is at least t.

Write exactly one line per interval command:

- `ADD`: print the resulting interval AVL tree in the nested-parentheses format defined below;
- `REMOVE`: after a successful removal, print the resulting tree; print `not found` if the ID is absent;
- `UPDATE`: after a successful update, print the resulting tree; print `not found` if the ID is absent;
- `CONFLICT`: print `yes` or `no`;
- `OVERLAPS` and `AT`: print matching IDs separated by one space in increasing (start, id) order, or print `none` when there is no match; and
- `NEXT`: print `id start end`, or print `none` when no event starts at or after the requested time.

Use the same recursive nested-parentheses format as for the ordinary AVL tree, but print only the generated event ID as each node label. Thus, a leaf with ID 7 is `7`, and a node with ID 4, an empty left child, and a right child with ID 9 is `4(,9)`. The representation shows the actual topology of the interval AVL tree, whose ordering key remains (start, id); do not print start times, end times, heights, or maxEnd values in this representation. Both child positions and the comma are required for every non-leaf node. If a successful `REMOVE` makes the tree empty, print an empty line.
For `UPDATE`, keep the same event ID: first remove its old (start, id) key and then insert the new one. If the ID is absent, the scheduler and the ID counter remain unchanged.

Sample interval input

```
ADD 9 11
ADD 10 12
ADD 13 15
ADD 16 20
CONFLICT 11 13
OVERLAPS 11 17
AT 11
NEXT 12
UPDATE 1 20 25
REMOVE 2
OVERLAPS 8 14
```

Sample interval output

```
1
1(,2)
2(1,3)
2(1,3(,4))
yes
2 3 4
2
3 13 15
3(2,4(,1))
4(3,1)
3
```

2.5 Timing output

After the final command, write the timing summary to standard output, not to the operation-output file. Use nanoseconds and this comma-separated format:

```
operation,count,total_ns,average_ns
<operation>,<count>,<total>,<average>
...
```

The AVL rows are `insert`, `delete`, `find`, and `traverse`. The interval-scheduler rows are `add`, `remove`, `update`, `conflict`, `overlaps`, `at`, and `next`. Use `N/A` for the average when the corresponding count is zero. Measure only the data-structure method. For result-producing queries, include construction of the returned result but exclude formatting and file output.

3 Assignment Tasks

3.1 Task 1: AVL Tree Implementation (40 marks)

Understand and implement the ordinary integer-key AVL module from first principles.

- Maintain each node's height and compute its balance factor correctly.
- Implement LL, RR, LR, and RL repairs for insertion.
- Implement deletion and continue rebalancing toward the root.
- Support Find, Traverse, duplicate detection, and all specified output.

3.2 Task 2: Interval Scheduling System (30 marks)

Build an augmented interval tree using your AVL balancing implementation.

- Order event nodes by (start, id) and maintain maxEnd at every node.
- Keep height and maxEnd correct after insertion, deletion, successor replacement, and every rotation.
- Implement Add, Remove, and Update without an ordered-container library.
- Implement Conflict, Overlaps, At, and Next with the specified pruning, deterministic ordering, and asymptotic costs.

3.3 Task 3: Timing Report (20 marks)

Instrument both programs using `std::chrono::steady_clock` or an equivalent monotonic high-resolution timer. Run `testcase.txt` with the ordinary AVL program and `interval_testcase_large.txt` with the interval scheduler, starting each program with an empty tree. Create a plain-text file named `timing_report.txt` containing the complete comma-separated timing summary produced by each program, clearly labeled `AVL` and `Interval Scheduler`.
This is a timing record, not a benchmark or a comparison between alternative data structures. Do not include input parsing, tree serialization, file I/O, console output, or debug

validation in the measured duration. Timing values may vary between executions; report the values from one complete valid run of each supplied testcase.

3.4 Task 4: Proper Submission (10 marks)

Create a folder named with your seven-digit student ID, for example `2405xxx`. It must contain:

- all source files for the ordinary AVL tree and interval scheduler;
- any required custom header files;
- `timing_report.txt`; and
- a short `README.txt` containing the exact compilation command and one example command for each of the two programs.

Compress the folder as `2405xxx.zip` and upload that single ZIP file to Moodle. Do not submit executables, object files, build directories, editor metadata, supplied testcases, or other generated artifacts.

Grading Breakdown

Component	Marks
Task 1: Implement an AVL tree	40
Task 2: Build the AVL-based interval scheduling system	30
Task 3: Create the timing report	20
Task 4: Proper submission	10
Total	**100**

Important Notes

Read Before You Submit

- AVL Search, Insert, and Delete must run in $O(\log n)$ worst-case time; Traverse must run in $O(n)$ time.
- Interval Add, Remove, Update, Conflict, and Next must run in $O(\log n)$ expected or worst-case time as specified. Overlaps and At must use maxEnd-based pruning, although reporting queries may still require $O(n)$ time in the worst case.
- Your program will be tested with operation orders designed to trigger every rotation, deletion repair, maxEnd update, endpoint case, and query-pruning case.
- Code must compile using the command in your README and must not depend on absolute paths or a particular IDE.
- Follow the course academic-integrity policy. You must be able to explain and modify every part of your submission during evaluation or viva.
- **DO NOT** copy code from others, plagiarize, or use generative AI for writing code. If the usage of such unfair means is proven, the submission will receive a **negative 100% (-100)** score, and **other appropriate departmental actions** will be taken.
- Questions should first be posted in the designated Moodle discussion thread so that clarifications are visible to everyone.

Acknowledgement

The visual template used for this handout was originally designed by **Md. Ashrafur Rahman Khan**, Lecturer, Department of Computer Science and Engineering, BUET, and has been adapted for this offering of CSE 208. We gratefully acknowledge this contribution.